

Experiment No.: 1

Title: Simulation of Register Transfer Operations and ALU Operations

Aim: To design and simulate register transfer operations and basic arithmetic and logic operations using multiple registers and an ALU.

Software/Tools Required

- Programming Language: C++
- Compiler: [OnlineGDB C++ Compiler](#)

Procedure

1. Start the program.
2. Initialize registers R1 and R2 with input values.
3. Transfer the contents of R1 and R2 to temporary registers.
4. Perform addition using the ALU.
5. Perform subtraction using the ALU.
6. Perform bitwise AND.
7. Perform bitwise OR.
8. Store the results in destination registers.
9. Display all register values and ALU results.
10. Stop.

Code :

```
#include <iostream>

using namespace std;

int main() {

    int R1 = 20;

    int R2 = 10;

    int R3, R4, R5, R6;

    cout << "REGISTER TRANSFER AND ALU SIMULATION\n";

    cout << "-----\n";

    cout << "Initial Registers:\n";

    cout << "R1 = " << R1 << endl;

    cout << "R2 = " << R2 << endl;
```

```

// Register transfer

R3 = R1;

C out << "\nRegister Transfer:\n";

cout << "R1 -> R3 : R3 = " << R3 << endl;

R4 = R2;

cout << "R2 -> R4 : R4 = " << R4 << endl;

// ALU operations

R5 = R1 + R2;

cout << "\nALU Operations:\n";

cout << "Addition : R1 + R2 = " << R5 << endl;

R5 = R1 - R2;

cout << "Subtraction: R1 - R2 = " << R5 << endl;

R5 = R1 & R2;

cout << "AND      : R1 & R2 = " << R5 << endl;

R6 = R1 | R2;

cout << "OR       : R1 | R2 = " << R6 << endl;

cout << "\nExecution completed successfully.";

return 0;

}

```

The screenshot shows a C++ IDE with the source code on the left and the execution output on the right. The source code is a C++ program that simulates register transfer and ALU operations. The output window shows the results of the simulation, including initial register values, register transfer operations, and ALU operations.

```

main.cpp
11 int main() {
12     int R1 = 20;
13     int R2 = 10;
14
15     int R3, R4, R5, R6;
16
17     cout << "REGISTER TRANSFER AND ALU SIMULATION\n";
18     cout << "-----\n";
19
20     cout << "Initial Registers:\n";
21     cout << "R1 = " << R1 << endl;
22     cout << "R2 = " << R2 << endl;
23
24     // Register transfer
25     R3 = R1;
26     cout << "\nRegister Transfer:\n";
27     cout << "R1 -> R3 : R3 = " << R3 << endl;
28
29     R4 = R2;
30     cout << "R2 -> R4 : R4 = " << R4 << endl;
31
32     // ALU operations
33     R5 = R1 + R2;
34     cout << "\nALU Operations:\n";
35     cout << "Addition : R1 + R2 = " << R5 << endl;
36
37     R5 = R1 - R2;
38     cout << "Subtraction: R1 - R2 = " << R5 << endl;
39
40     R5 = R1 & R2;
41     cout << "AND      : R1 & R2 = " << R5 << endl;
42
43     R6 = R1 | R2;
44     cout << "OR       : R1 | R2 = " << R6 << endl;
45
46     cout << "\nExecution completed successfully.";
47

```

```

REGISTER TRANSFER AND ALU SIMULATION
-----
Initial Registers:
R1 = 20
R2 = 10

Register Transfer:
R1 -> R3 : R3 = 20
R2 -> R4 : R4 = 10

ALU Operations:
Addition : R1 + R2 = 30
Subtraction: R1 - R2 = 10
AND      : R1 & R2 = 0
OR       : R1 | R2 = 30

Execution completed successfully.
...Program finished with exit code 0
Press ENTER to exit console.

```

Experiment No.: 2

Title: Simulation and Comparison of Single-Bus and Multi-Bus Organization

Aim: To simulate single-bus and multi-bus computer organization and compare their data transfer efficiency.

Tools Required

- Programming Language: C++
- Compiler: [OnlineGDB C++ Compiler](#)

Procedure

1. Initialize registers and memory.
2. Simulate data transfer using a single bus.
3. Count the number of transfer cycles.
4. Simulate data transfer using multiple buses.
5. Allow simultaneous transfers where possible.
6. Calculate total cycles for both systems.
7. Compare the results.
8. Display the more efficient architecture.

code

```
#include <iostream>

using namespace std;

int main() {

    int R1 = 100;

    int R2 = 200;

    int R3 = 0;

    int Memory = 500;

    int IO = 50;

    cout << "SINGLE-BUS AND MULTI-BUS ORGANIZATION\n";

    cout << "-----\n";

    cout << "\nSINGLE-BUS OPERATION\n";

    int singleBusCycles = 0;

    R3 = R1;

    singleBusCycles++;

    cout << "R1 -> R3 : " << R3 << endl;

    R3 = R2;

    singleBusCycles++;

    cout << "R2 -> R3 : " << R3 << endl;
```

```

R3 = Memory;

singleBusCycles++;

cout << "Memory -> R3 : " << R3 << endl;

R3 = IO;

singleBusCycles++;

cout << "I/O -> R3 : " << R3 << endl;

cout << "Total Single-Bus Cycles = "
    << singleBusCycles << endl;

// Multi-bus simulation

cout << "\nMULTI-BUS OPERATION\n";

int multiBusCycles = 0;

cout << "Bus 1: R1 -> R3 : " << R1 << endl;

cout << "Bus 2: R2 -> R3 : " << R2 << endl;

multiBusCycles++;

cout << "Bus 1: Memory -> R3 : " << Memory << endl;

cout << "Bus 2: I/O -> R3 : " << IO << endl;

multiBusCycles++;

cout << "Total Multi-Bus Cycles = "
    << multiBusCycles << endl;

cout << "\nPERFORMANCE COMPARISON\n";

cout << "Single-Bus Cycles = " << singleBusCycles << endl;

cout << "Multi-Bus Cycles = " << multiBusCycles << endl;

double improvement =
    ((double)(singleBusCycles - multiBusCycles)
    / singleBusCycles) * 100;

cout << "Cycle Reduction = "
    << improvement << "%" << endl;

return 0;

}

```

```

30 R3 = IO;
31 singleBusCycles++;
32 cout << "I/O -> R3 : " << R3 << endl;
33
34 cout << "Total Single-Bus Cycles = "
35 << singleBusCycles << endl;
36
37 // Multi-bus simulation
38 cout << "\nMULTI-BUS OPERATION\n";
39
40 int multiBusCycles = 0;
41
42 cout << "Bus 1: R1 -> R3 : " << R1 << endl;
43 cout << "Bus 2: R2 -> R3 : " << R2 << endl;
44 multiBusCycles++;
45
46 cout << "Bus 1: Memory -> R3 : " << Memory << endl;
47 cout << "Bus 2: I/O -> R3 : " << IO << endl;
48 multiBusCycles++;
49
50 cout << "Total Multi-Bus Cycles = "
51 << multiBusCycles << endl;
52
53 cout << "\nPERFORMANCE COMPARISON\n";
54 cout << "Single-Bus Cycles = " << singleBusCycles << endl;
55 cout << "Multi-Bus Cycles = " << multiBusCycles << endl;
56
57 double improvement =
58 ((double)(singleBusCycles - multiBusCycles)
59 / singleBusCycles) * 100;
60
61 cout << "Cycle Reduction = "
62 << improvement << "%" << endl;
63
64 return 0;
65
66

```

```

SINGLE-BUS OPERATION
R1 -> R3 : 100
R2 -> R3 : 200
Memory -> R3 : 500
I/O -> R3 : 50
Total Single-Bus Cycles = 4

MULTI-BUS OPERATION
Bus 1: R1 -> R3 : 100
Bus 2: R2 -> R3 : 200
Bus 1: Memory -> R3 : 500
Bus 2: I/O -> R3 : 50
Total Multi-Bus Cycles = 2

PERFORMANCE COMPARISON
Single-Bus Cycles = 4
Multi-Bus Cycles = 2
Cycle Reduction = 50%

...Program finished with exit code 0
Press ENTER to exit console.

```

Experiment No.: 3

Title: Simulation of a 5-Stage Instruction Pipeline and Performance Analysis

Aim: To simulate a five-stage instruction pipeline and calculate its speedup and throughput compared with a non-pipelined processor.

Tools Required

- Programming Language: C++
- Compiler: [OnlineGDB C++ Compiler](#)

```
#include <iostream>
```

```
#include <iomanip>
```

```
using namespace std;
```

```
int main() {
```

```
    int instructions = 8;
```

```
    int stages = 5;
```

```
    // Non-pipelined execution
```

```
    int nonPipelinedCycles = instructions * stages;
```

```
    // Pipelined execution
```

```
    int pipelinedCycles = stages + instructions - 1;
```

```
    double speedup =
```

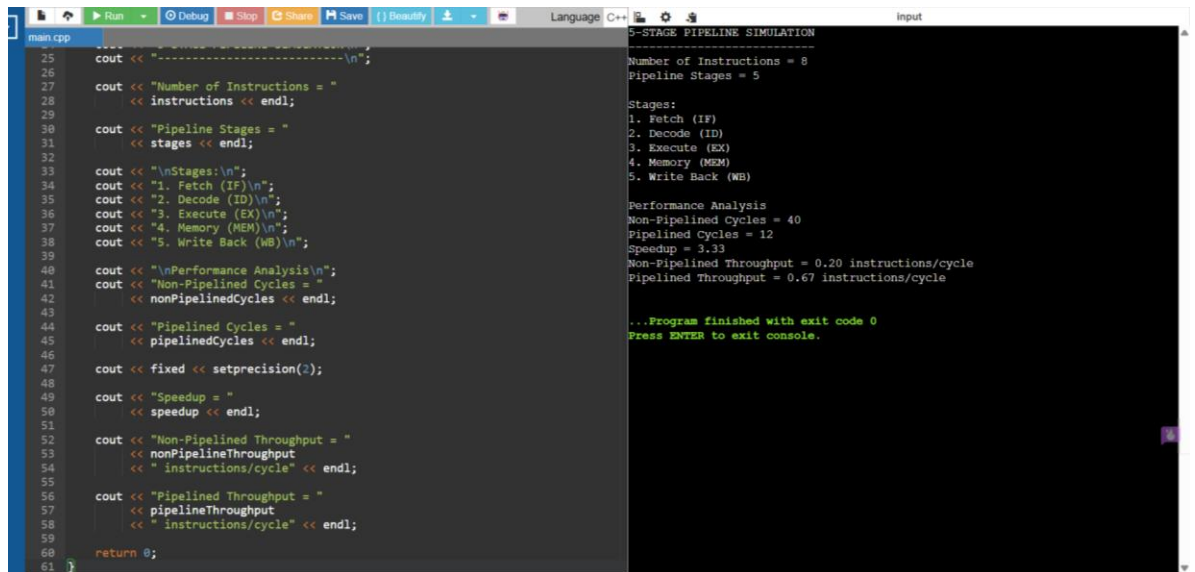
```
        (double)nonPipelinedCycles / pipelinedCycles;
```

```
    double nonPipelineThroughput =
```

```

        (double)instructions / nonPipelinedCycles;
double pipelineThroughput =
    (double)instructions / pipelinedCycles;
cout << "5-STAGE PIPELINE SIMULATION\n";
cout << "-----\n";
cout << "Number of Instructions = "
    << instructions << endl;
cout << "Pipeline Stages = "
    << stages << endl;
cout << "\nStages:\n";
cout << "1. Fetch (IF)\n";
cout << "2. Decode (ID)\n";
cout << "3. Execute (EX)\n";
cout << "4. Memory (MEM)\n";
cout << "5. Write Back (WB)\n";
cout << "\nPerformance Analysis\n";
cout << "Non-Pipelined Cycles = "
    << nonPipelinedCycles << endl;
cout << "Pipelined Cycles = "
    << pipelinedCycles << endl;
cout << fixed << setprecision(2);
cout << "Speedup = "
    << speedup << endl;
cout << "Non-Pipelined Throughput = "
    << nonPipelineThroughput
    << " instructions/cycle" << endl;
cout << "Pipelined Throughput = "
    << pipelineThroughput
    << " instructions/cycle" << endl;
return 0;
}

```



```
main.cpp 25 cout << "-----\n";
26 cout << "Number of Instructions = "
27 << instructions << endl;
28 cout << "Pipeline Stages = "
29 << stages << endl;
30 cout << "\nStages:\n";
31 cout << "1. Fetch (IF)\n";
32 cout << "2. Decode (ID)\n";
33 cout << "3. Execute (EX)\n";
34 cout << "4. Memory (MEM)\n";
35 cout << "5. Write Back (WB)\n";
36
37 cout << "\nPerformance Analysis\n";
38 cout << "Non-Pipelined Cycles = "
39 << nonPipelinedCycles << endl;
40 cout << "Pipelined Cycles = "
41 << pipelinedCycles << endl;
42 cout << fixed << setprecision(2);
43 cout << "Speedup = "
44 << speedup << endl;
45 cout << "Non-Pipelined Throughput = "
46 << nonPipelineThroughput
47 << " instructions/cycle" << endl;
48 cout << "Pipelined Throughput = "
49 << pipelineThroughput
50 << " instructions/cycle" << endl;
51
52 return 0;
53 }
```

```
5-STAGE PIPELINE SIMULATION
Number of Instructions = 8
Pipeline Stages = 5

Stages:
1. Fetch (IF)
2. Decode (ID)
3. Execute (EX)
4. Memory (MEM)
5. Write Back (WB)

Performance Analysis
Non-Pipelined Cycles = 40
Pipelined Cycles = 12
Speedup = 3.33
Non-Pipelined Throughput = 0.20 instructions/cycle
Pipelined Throughput = 0.67 instructions/cycle

...Program finished with exit code 0
Press ENTER to exit console.
```

Experiment No.: 4

Title: Simulation of Data, Control and Structural Hazards

Aim

To simulate different types of pipeline hazards and study techniques used to minimize their performance impact.

Tools Required

- Programming Language: C++
- Compiler: [OnlineGDB C++ Compiler](#)

Procedure

1. Define a sequence of instructions.
2. Check for data dependencies.
3. Introduce a data hazard.
4. Resolve it using forwarding.
5. Introduce a branch instruction.
6. Simulate a control hazard.
7. Resolve it using branch prediction.
8. Introduce resource conflict.
9. Simulate a structural hazard.

```

#include <iostream>

using namespace std;

int main() {

    cout << "PIPELINE HAZARD SIMULATION\n";

    cout << "=====\n";

    int baseCycles = 10;

    // Data Hazard

    cout << "\n1. DATA HAZARD\n";

    cout << "ADD R1, R2, R3\n";

    cout << "SUB R4, R1, R5\n";

    cout << "Hazard detected: SUB needs R1 from ADD.\n";

    cout << "Resolution: Data forwarding.\n";

    int dataStall = 0

    // Control Hazard

    cout << "\n2. CONTROL HAZARD\n";

    cout << "BEQ R1, R2, TARGET\n";

    cout << "Next instruction depends on branch result.\n";

    cout << "Resolution: Branch prediction.\n";

    int controlStall = 1;

    // Structural Hazard

    cout << "\n3. STRUCTURAL HAZARD\n";

    cout << "Instruction Fetch and Memory Access require\n";

    cout << "the same memory resource.\n";

    cout << "Resolution: Insert one stall cycle.\n";

    int structuralStall = 1;

    int totalWithHazards =

        baseCycles + 2 + 3;

    int optimizedCycles =

        baseCycles + dataStall

        + controlStall

        + structuralStall;  cout << "\nPERFORMANCE ANALYSIS\n";

```

```

cout << "Base Pipeline Cycles = "

    << baseCycles << endl;

cout << "Cycles with Hazards = "

    << totalWithHazards << endl;

cout << "Cycles after Hazard Resolution = "

    << optimizedCycles << endl;

cout << "\nHazard reduction = "

    << totalWithHazards - optimizedCycles

    << " cycles" << endl;

cout << "\nSimulation completed successfully.";

return 0;

}

```

```

main.cpp
25
26 int controlStall = 1;
27 // Structural Hazard
28 cout << "\n3. STRUCTURAL HAZARD\n";
29 cout << "Instruction Fetch and Memory Access require\n";
30 cout << "The same memory resource.\n";
31 cout << "Resolution: Insert one stall cycle.\n";
32
33 int structuralStall = 1;
34
35 int totalWithHazards =
36     baseCycles + 2 + 3;
37
38 int optimizedCycles =
39     baseCycles + dataStall
40     + controlStall
41     + structuralStall;
42
43 cout << "\nPERFORMANCE ANALYSIS\n";
44 cout << "Base Pipeline Cycles = "
45     << baseCycles << endl;
46
47 cout << "Cycles with Hazards = "
48     << totalWithHazards << endl;
49
50 cout << "Cycles after Hazard Resolution = "
51     << optimizedCycles << endl;
52
53 cout << "\nHazard reduction = "
54     << totalWithHazards - optimizedCycles
55     << " cycles" << endl;
56
57 cout << "\nSimulation completed successfully.";
58
59 return 0;
60
61 }

```

```

PIPELINE HAZARD SIMULATION
1. DATA HAZARD
ADD R1, R2, R3
SUB R4, R1, R5
Hazard detected: SUB needs R1 from ADD.
Resolution: Data forwarding.
2. CONTROL HAZARD
BEQ R1, R2, TARGET
Next instruction depends on branch result.
Resolution: Branch prediction.
3. STRUCTURAL HAZARD
Instruction Fetch and Memory Access require
the same memory resource.
Resolution: Insert one stall cycle.
PERFORMANCE ANALYSIS
Base Pipeline Cycles = 10
Cycles with Hazards = 15
Cycles after Hazard Resolution = 12
Hazard reduction = 3 cycles
Simulation completed successfully.
...Program finished with exit code 0
Press ENTER to exit console.

```

Experiment No.: 5

Title: Simulation of Superscalar, Out-of-Order, Speculative Execution and SMT

Aim

To simulate superscalar processor execution and compare normal, out-of-order, speculative and simultaneous multithreaded execution.

Tools Required

- Programming Language: C++
- Compiler: [OnlineGDB C++ Compiler](#)

Procedure

1. Define a set of instructions.
2. Execute instructions using a single-issue processor.
3. Calculate the number of cycles.
4. Simulate a 2-wide superscalar processor.
5. Execute independent instructions in parallel.
6. Simulate out-of-order execution.
7. Simulate speculative execution.
8. Simulate two threads using SMT.
9. Calculate instruction throughput.
10. Compare all architectures.

```
main.cpp
62 // ...
63 cout << "\nInstruction Throughput\n";
64 cout << "Single Issue          = "
65     << singleThroughput << " instr/cycle" << endl;
66
67 cout << "Superscalar          = "
68     << superThroughput << " instr/cycle" << endl;
69
70 cout << "Out-of-Order          = "
71     << oooThroughput << " instr/cycle" << endl;
72
73 cout << "Speculative          = "
74     << speculativeThroughput << " instr/cycle" << endl;
75
76 cout << "SMT          = "
77     << smtThroughput << " instr/cycle" << endl;
78
79 cout << "\nPerformance Improvement\n";
80
81 cout << "Superscalar Speedup = "
82     << ((double)singleIssueCycles /
83         superscalarCycles << "x" << endl;
84
85 cout << "Out-of-Order Speedup = "
86     << ((double)singleIssueCycles /
87         outOfOrderCycles << "x" << endl;
88
89 cout << "Speculative Speedup = "
90     << ((double)singleIssueCycles /
91         speculativeCycles << "x" << endl;
92
93 cout << "SMT Speedup = "
94     << ((double)singleIssueCycles /
95         smtCycles << "x" << endl;
96
97 return 0;
98 }
```

```
SUPERSCALAR AND SMT PROCESSOR SIMULATION
=====
Number of Instructions = 12

Execution Cycles
Single Issue          = 12
2-Wide Superscalar   = 6
Out-of-Order         = 5
Speculative Execution = 4
SMT                   = 3

Instruction Throughput
Single Issue          = 1.00 instr/cycle
Superscalar          = 2.00 instr/cycle
Out-of-Order         = 2.40 instr/cycle
Speculative          = 3.00 instr/cycle
SMT                   = 4.00 instr/cycle

Performance Improvement
Superscalar Speedup = 2.00x
Out-of-Order Speedup = 2.40x
Speculative Speedup = 3.00x
SMT Speedup = 4.00x

...Program finished with exit code 0
Press ENTER to exit console.
```